現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

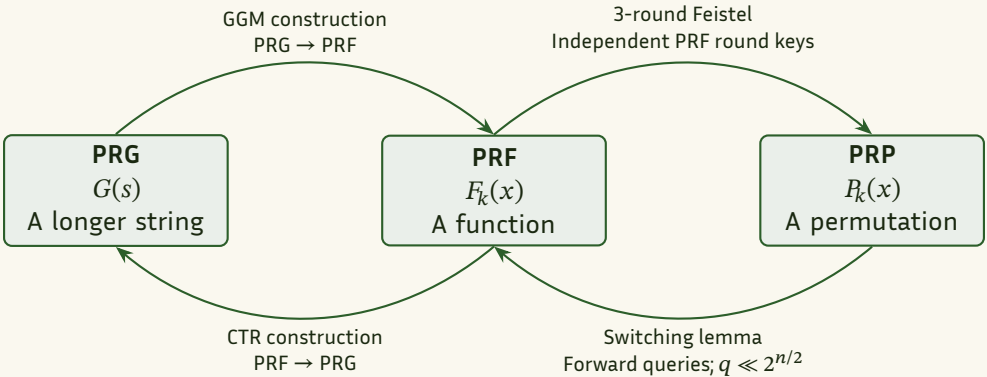
1.4: Pseudorandomness

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.

PRG, PRF, and PRP: construction relationships



Arrows describe constructions or security reductions, not set inclusion.

Here n is the PRP block length; q counts forward queries.

Section 1

Pseudorandom Generators

Limitations of One-Time Pad

The Key Length Problem

One-time pad requires as much shared key material as message data.

- The key must be as long as the plaintext!
- This creates a chicken-and-egg situation:
  - To privately send n bits of information,
  - We must already privately share n bits of information.
- Impractical for most real-world applications.
  - Clearly this is not what we're doing when we use HTTPS,
  - or WhatsApp, or pay for something via a debit card...
- We need encryption schemes where the key can be smaller than the message.

Idea: find a way to expand the key

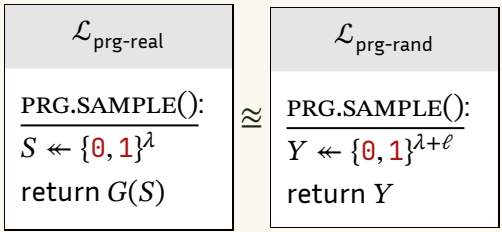
- Given a key k_s of size $|k_s| < |m|$, find a way to obtain a key k_e where $|k_e| \geq |m|$.
- In the real world, we have two kinds of symmetric encryption schemes:
  - **Block ciphers**: AES.
  - **Stream ciphers**: ChaCha20.
- This is exactly what stream ciphers do!
  - Start with a small key k_s of a fixed size $|k_s| = \lambda$,
  - Deterministically expand it to k_e where $|k_e| \geq |m|$,
  - $\text{ENC}(k_e, M) = k_e \oplus M$

Requirements for Key Expansion

- Our method would need to be **deterministic**, so that both the sender and receiver can expand their key in the same way (to encrypt/decrypt).
- Its output distribution would need to be **uniform**, since that is a crucial property for the security of OTP.
- Unfortunately, it's **not possible** to achieve both of these properties simultaneously.
  - Suppose the expansion method is a deterministic function $G : \{0,1\}^n \to \{0,1\}^{n+\ell}$.
  - Its outputs are ℓ bits longer than its inputs!
  - There are $2^{n+\ell}$ strings of length $n+\ell$ but only (at most) 2^n possible outputs of G.
  - So the outputs of G can never induce a uniform distribution.

Enter pseudorandomness

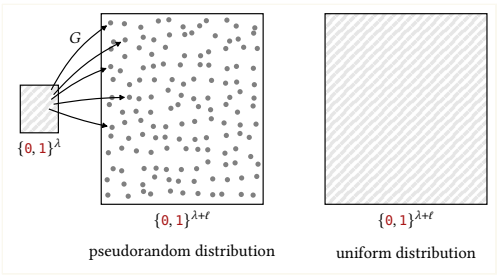
- In cryptography, something is **pseudorandom** if it is indistinguishable from a uniform distribution.[a]
- We need to invent some **"secure pseudorandom generator" (PRG)** G that takes a **seed** S and ends up being indistinguishable from a true uniform distribution when thrown into a library:



[a] The Oxford English Dictionary defines the prefix pseudo- as "apparently but not really."

Enter pseudorandomness

- In cryptography, something is **pseudorandom** if it is indistinguishable from a uniform distribution.[a]
- We need to invent some **"secure pseudorandom generator" (PRG)** G that takes a **seed** S and ends up being indistinguishable from a true uniform distribution when thrown into a library:

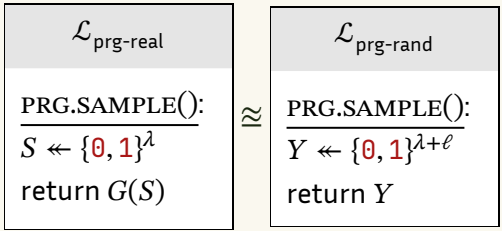


Source: The Joy of Cryptography

[a] The Oxford English Dictionary defines the prefix pseudo- as "apparently but not really."

Enter pseudorandomness

- We have practical candidates and constructions based on assumptions, but no unconditional proof that secure PRGs exist.
- Statistical tests alone do not establish security against every efficient distinguisher.
- The concept of **computational indistinguishability** with **negligible difference** is again useful here!



Perfect one-time secrecy of an SKE

Perfect One-time Secrecy for SKE

An SKE scheme Σ has perfect one-time secrecy if the following libraries are interchangeable:

$\mathcal{L}^{\Sigma}_{\text{ots-real}}$

$\text{OTS.ENC}(M)$:
$K \twoheadleftarrow \Sigma.\mathcal{K}$
$C \coloneqq \Sigma.\text{Enc}(K, M)$
return C

$\equiv$

$\mathcal{L}^{\Sigma}_{\text{ots-rand}}$

$\text{OTS.ENC}(M)$:
$C \twoheadleftarrow \Sigma.\mathcal{C}$
return C

An encryption scheme has perfect one-time secrecy if its ciphertexts are uniformly distributed when keys are sampled uniformly, kept secret, and used for only one encryption, no matter how the plaintexts are chosen.

Computational one-time secrecy of pseudo-OTP

If:

$\mathcal{L}_{\text{prg-real}}$

$\text{PRG.SAMPLE}()$:
$S \twoheadleftarrow \{0,1\}^{\lambda}$
return $G(S)$

$\approx$

$\mathcal{L}_{\text{prg-rand}}$

$\text{PRG.SAMPLE}()$:
$Y \twoheadleftarrow \{0,1\}^{\lambda+\ell}$
return Y

Then:

$\mathcal{L}_{\text{ots-real}}$

$\text{OTS.ENC}(M)$:
$K \twoheadleftarrow \{0,1\}^{\lambda}$
$Y := G(K)$
$C := Y \oplus M$
return C

$\approx$

$\mathcal{L}_{\text{ots-rand}}$

$\text{OTS.ENC}(M)$:
$C \twoheadleftarrow \{0,1\}^{\lambda+\ell}$
return C

Pseudo-OTP: a short hybrid proof

Let $K \leftarrow \{0,1\}^\lambda$ and let U be an independent uniform $(\lambda + \ell)$-bit string.

$$\underbrace{M \oplus G(K)}_{\text{Pseudo-OTP}} \underset{\text{PRG security}}{\approx} \underbrace{M \oplus U}_{\text{True OTP}} \underset{\text{OTP property}}{\equiv} \underbrace{U}_{\text{Uniform ciphertext}}.$$

- First replace the PRG output with fresh uniform randomness.
- XOR with a fixed message preserves the uniform distribution.
- $\approx$: computational indistinguishability. $\equiv$: identical distributions.

The guarantee is conditional

If G is secure, pseudo-OTP has computational one-time secrecy.

Source: The Joy of Cryptography.

Pseudo-OTP: the mask is still one-time

$$C_1 = M_1 \oplus G(K), \qquad C_2 = M_2 \oplus G(K)$$

$$\boxed{C_1 \oplus C_2 = M_1 \oplus M_2}$$

- Expanding a short key does not make the resulting mask reusable.
- If M_1 is known, an attacker recovers $G(K)$ and then M_2.
- Practical stream encryption separates messages using a nonce or maintained state.
- The key, nonce, and counter must not reproduce the same keystream positions.

Attacking PRGs

- Assume that G is a secure PRG.
- Is $\mathrm{H}(S)$ secure?

$\mathrm{H}(S)$:
$A \| B := G(S)$
$C \| D := G(B)$
return $A \| B \| C \| D$

$\mathcal{L}^{H}_{\text{prg-real}}$

PRG.SAMPLE():
$S \twoheadleftarrow \{0,1\}^{\lambda}$
$A \| B := G(S)$
$C \| D := G(B)$
return $A \| B \| C \| D$

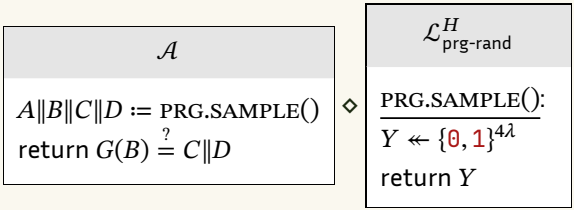
$\overset{?}{\approx}$

$\mathcal{L}^{H}_{\text{prg-rand}}$

PRG.SAMPLE():
$Y \twoheadleftarrow \{0,1\}^{4\lambda}$
return Y

Attacking PRGs

- Assume that G is a secure PRG.
- Is $\mathrm{H}(S)$ secure?
- **No**:
- $\Pr\left[\mathcal{A} \diamond \mathcal{L}^{H}_{\text{prg-real}} \Rightarrow \texttt{true}\right] = 1$
- $\Pr\left[\mathcal{A} \diamond \mathcal{L}^{H}_{\text{prg-rand}} \Rightarrow \texttt{true}\right] = \frac{1}{2^{2\lambda}}$
- This difference is certainly not negligible.



Extending a PRG securely

$\underline{\mathrm{H}(S)}$:
$A \| B := G(S)$
$C \| D := G(B)$
return $A \| C \| D$

- Start with $G : \{0,1\}^{\lambda} \to \{0,1\}^{2\lambda}$.
- The new output has length 3λ.
- Keep the intermediate seed B hidden.
- Revealing B would allow the check $G(B) = C \| D$ used in our attack.

Security claim

If G is a secure length-doubling PRG, this H is a secure length-tripling PRG.

Source: The Joy of Cryptography.

Why the PRG extension is secure

1. Real construction: sample S and return $A\|G(B)$, where $A\|B = G(S)$.
2. Replace $G(S)$ with independent uniform A, B using PRG security.
3. Now B is a hidden uniform seed. Replace $G(B)$ with an independent uniform 2λ-bit string.

$$A\|G(B) \approx U_\lambda\|G(U'_\lambda) \approx U_{3\lambda}.$$

The order matters

Do not assume every internal call to G has a uniform seed. Establish that condition before replacing its output.

Source: The Joy of Cryptography.

Example: RC4 (a stream cipher)

- Used in WEP, SSL/TLS, and other protocols.
- Historical keystream generator; not a secure PRG by our definition.
- Basic operation:
  - Initialize the S-box with a permutation of bytes 0–255.
  - Use the key to scramble the S-box.
  - Generate pseudorandom bytes iteratively.
- Several weaknesses found over time:
  - Statistical biases in initial output.
  - Correlation between key and output bytes.
  - Considered cryptographically broken today.

Security Warning

RC4 is presented as a historical example only. It should not be used in new applications due to known weaknesses. Modern alternatives include ChaCha20.

RC4 PRG Algorithm

- RC4 generates a pseudorandom stream of bytes used to encrypt data.
- After key setup (which initializes array S), the algorithm produces keystream bytes.
- Each output byte requires simple operations:
  - Array index calculations.
  - Array value swapping.
  - Modular addition.
- Fast implementation in software.
- Despite simplicity, it has several cryptographic weaknesses.

```python
def PRGA(S):
    i = 0
    j = 0
    while True:
        i = (i + 1) % 256
        j = (j + S[i]) % 256

        S[i], S[j] = S[j], S[i]
        K = S[(S[i] + S[j]) % 256]
        yield K
```

RC4 PRG implementation in Python.

Section 2

Pseudorandom Functions

Pseudorandom function: definition

Pseudorandom Function (PRF)

A function $F : \{0,1\}^\lambda \times \{0,1\}^n \to \{0,1\}^m$ is a secure pseudorandom function (PRF) if the following two libraries are indistinguishable:

- n is the input length of the PRF.
- m is the output length of the PRF.
- λ is the key size and hence the security parameter.

$\mathcal{L}^F_{\text{prf-real}}$

$K \twoheadleftarrow \{0,1\}^\lambda$

$\underline{\text{PRF.QUERY}(X)}$:

return $F(K, X)$

$\approx$

$\mathcal{L}^F_{\text{prf-rand}}$

$L := [\,]$

$\underline{\text{PRF.QUERY}(X)}$:

if $L[X]$ undefined:

$L[X] \twoheadleftarrow \{0,1\}^m$

return $L[X]$

PRG versus PRF: Key Differences

- **PRG (Pseudorandom Generator)**:
  - Takes a short seed, produces a longer output.
  - Generates the entire output as a monolithic string.
  - $G : \{0,1\}^{\lambda} \to \{0,1\}^{\lambda+\ell}$
- **PRF (Pseudorandom Function)**:
  - Maps inputs to pseudorandom outputs.
  - Provides access to individual blocks of output.
  - Can generate output for any input on demand.
  - $F : \{0,1\}^{\lambda} \times \{0,1\}^{n} \to \{0,1\}^{m}$
- PRFs enable “selective access” to pseudorandom values without generating the entire sequence.

PRG and PRF: who chooses what?

	PRG game	PRF game
Secret	Fresh seed per sample	One key for all queries
Attacker input	No chosen seed	Chooses x adaptively
Attacker observes	$G(s)$	$F_k(x)$
Ideal response	Fresh uniform string	A consistent random table

Repeated PRF queries are allowed

Both worlds repeat answers to the same input. Only new entries are sampled independently.

Source: The Joy of Cryptography.

A PRF is a keyed, deterministic function

$$F : \{0,1\}^\lambda \times \{0,1\}^n \longrightarrow \{0,1\}^m, \qquad F_k(x) = F(k,x).$$

Fix the key

The mapping is deterministic. Repeating x repeats $F_k(x)$.

- The domain has 2^n elements.
- The codomain has 2^m elements.

Hide the key

Across chosen queries, the mapping should behave like a random function.

- Invertibility is not required.
- Collisions are allowed; if $n > m$, they are unavoidable.

An insecure PRF construction

Claim: An Insecure PRF

The function $F(K, X) = G(K) \oplus X$ is not a secure PRF, even if G is a secure PRG.

- This construction fails because:
  - The key K is only fed through the PRG once.
  - The same value $G(K)$ is used for all queries.
  - This creates exploitable patterns in outputs.

$\mathcal{L}^{F}_{\text{prf-real}}$

$K \twoheadleftarrow \{0,1\}^{\lambda}$

$\underline{\text{PRF.QUERY}(X)}$:

return $G(K) \oplus X$

$\approx$

$\mathcal{L}^{F}_{\text{prf-rand}}$

$L := [\,]$

$\underline{\text{PRF.QUERY}(X)}$:

if $L[X]$ undefined:

$L[X] \twoheadleftarrow \{0,1\}^{m}$

return $L[X]$

An insecure PRF construction

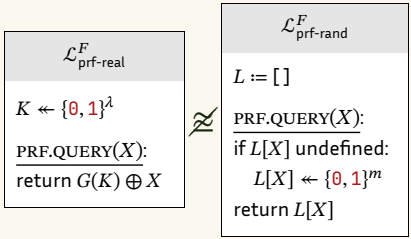
- When an adversary sees two outputs:

$$Y_1 = G(K) \oplus X_1$$
$$Y_2 = G(K) \oplus X_2$$

- Taking $Y_1 \oplus Y_2$ causes $G(K)$ to cancel:

$$\begin{aligned} Y_1 \oplus Y_2 &= G(K) \oplus X_1 \oplus G(K) \oplus X_2 \\ &= X_1 \oplus X_2 \end{aligned}$$

- With a truly random function,
  $Y_1 \oplus Y_2 = X_1 \oplus X_2$ would be extremely unlikely!

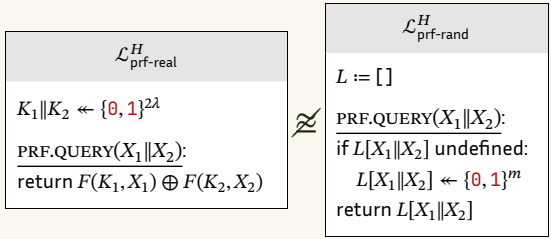


Another insecure PRF construction

Claim: Another Insecure PRF

The function $H(K_1 \| K_2, X_1 \| X_2) = F(K_1, X_1) \oplus F(K_2, X_2)$ is not a secure PRF, even if F is a secure PRF.

- An unsuccessful attempt to use a PRF with a shorter input length to build one with a larger input length.



Why the previous PRF is broken

- When inputs share the same first half, the corresponding outputs of H have a common term $F(K_1, X_1)$.
- Consider querying four inputs: $A\|B, A\|B'$, $A'\|B, A'\|B'$ (where $A \neq A'$ and $B \neq B'$)

$$
\begin{aligned}
Y_1 &= F(K_1, A) \oplus F(K_2, B) \\
Y_2 &= F(K_1, A) \oplus F(K_2, B') \\
Y_3 &= F(K_1, A') \oplus F(K_2, B) \\
Y_4 &= F(K_1, A') \oplus F(K_2, B')
\end{aligned}
$$

- If we XOR $Y_1 \oplus Y_2$, the $F(K_1, A)$ terms cancel out.
- Similarly, $Y_3 \oplus Y_4$ causes $F(K_1, A')$ to cancel:
  - $Y_1 \oplus Y_2 = F(K_2, B) \oplus F(K_2, B')$
  - $Y_3 \oplus Y_4 = F(K_2, B) \oplus F(K_2, B')$
- So $\Pr\left[Y_1 \oplus Y_2 = Y_3 \oplus Y_4\right] = 1$.
- With a truly random function, $\Pr\left[Y_1 \oplus Y_2 = Y_3 \oplus Y_4\right] = \frac{1}{2^m}$ (extremely unlikely!)
- Also, given $Y_{1\ldots3}$, we can predict Y_4!

The Golden Rule of PRFs

Fresh independent values need distinct inputs

When a construction needs fresh pseudorandom values under one key, it must avoid unintended reuse of PRF inputs.

- Repeated queries are allowed by the PRF definition: both worlds repeat their answers.
- An upper-level construction can fail if it treats a repeated output as fresh independent randomness.
- To attack a construction, look for inputs that force unintended reuse.
- To prove security, explain why the required internal inputs are distinct, or bound the probability of repetition.

Building a PRG from a PRF: CTR

$$G(s) = F_s(\langle 0 \rangle_n) \| F_s(\langle 1 \rangle_n) \| \cdots \| F_s(\langle t-1 \rangle_n).$$

- $\langle i \rangle_n$ is the fixed-width n-bit encoding of counter i.
- Use polynomially many blocks, $t \leq 2^n$, with $tm > \lambda$.
- Every counter input is distinct: do not wrap around.
- Replacing F_s by a random function gives independent uniform blocks.

The Golden Rule in action

Distinct counter inputs provide fresh table entries under the same PRF key.

Source: The Joy of Cryptography.

CTR security: one key at a time

- For one seed, a PRF distinguisher queries the counters and concatenates the answers.
- A PRF oracle gives the real CTR output; a random-function oracle gives a uniform string.
- Our PRG library allows many samples, each with a fresh seed.
- Use a hybrid for each sample: replace one seed's CTR output at a time.

$$\mathcal{H}_0 \approxeq \mathcal{H}_1 \approxeq \cdots \approxeq \mathcal{H}_q.$$

Use limitation: the same seed repeats the stream. CTR here is a PRG, not yet a complete encryption scheme.

Source: The Joy of Cryptography.

Building a PRF from a PRG: GGM intuition

Let $G(s) = G_0(s) \| G_1(s)$ be a secure length-doubling PRG.

$$s_0 = k, \qquad s_i = G_{x_i}(s_{i-1}) \quad (i = 1, \dots, n), \qquad F_k(x_1 \cdots x_n) = s_n.$$

- View each seed as a tree node; G creates its left and right children.
- Each input bit selects one branch. The key is the hidden root.
- An n-bit query follows one path with n PRG evaluations.
- The tree has exponentially many leaves, but we never build the whole tree.

Scope: PRG → PRF in theory; full proof optional.

Source: The Joy of Cryptography.

Section 3

Pseudorandom Permutations

What is a permutation?

Permutation

A permutation is a rearrangement where each input value maps to exactly one output value, and each possible output appears exactly once.

- Permutations rearrange elements rather than transforming them.
- Every element in the domain appears exactly once in the range.
- The function is invertible.

- **Example**: simple substitution cipher. Each letter maps to another letter:

$$a \mapsto g, \quad b \mapsto a, \quad c \mapsto r$$
$$d \mapsto b, \quad e \mapsto l, \quad \dots$$

- Under this permutation:
  - “cabbage” ↦ “rgaagdl”
  - “rgaagdl” ↦ “cabbage”
- This mapping is reversible because it’s a permutation over $\{a, \dots, z\}$.
- Each letter appears exactly once in the output alphabet.

PRF versus PRP

PRF

A keyed function $F_k : \{0,1\}^n \to \{0,1\}^m$.

- Imitates a random function.
- Input length is fixed in our basic definition.
- Distinct inputs may collide.
- No inverse is required.

PRP

A keyed permutation $P_k : \{0,1\}^n \to \{0,1\}^n$.

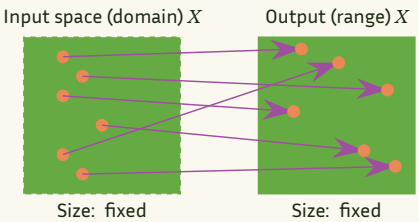
- Imitates a random permutation.
- Same input and output length.
- Distinct inputs never collide.
- Efficient inverse given the key.

A permutation is not automatically secure

- Invertibility is a correctness property; pseudorandomness is a security property.
- The identity mapping $P_k(x) = x$ is invertible and collision-free, but trivially distinguishable.
- No output collisions does not by itself prevent differential attacks or remove birthday limits in constructions.
- A PRP can act as a PRF for forward queries when the switching-lemma loss is sufficiently small.

PRP : $F_k : X \to X$

- **Bijective** (two-way)
  - **Injective**: No two inputs map to the same output (no collisions).
  - **Surjective**: Every output has one corresponding input.
- Deterministic for a fixed key
- Imitates a random permutation when the key is hidden



“Lazy dictionaries” versus “lazy permutations”

Ideal PRF versus ideal PRP

$\mathcal{L}^F_{\text{prf-rand}}$

$L := [\,]$

$\underline{\text{PRF.QUERY}(X)}$:
if $L[X]$ undefined:
  $L[X] \twoheadleftarrow \{0,1\}^m$
return $L[X]$

$\mathcal{L}^F_{\text{prp-rand}}$

$L := [\,]; \mathcal{Y} := \varnothing$

$\underline{\text{PRP.QUERY}(X)}$:
if $L[X]$ undefined:
  $Y \twoheadleftarrow \{0,1\}^n \setminus \mathcal{Y}$
  $\mathcal{Y} := \mathcal{Y} \cup \{Y\}$
  $L[X] := Y$
return $L[X]$

“Lazy dictionaries” versus “lazy permutations”

- While the PRF (on the left) just picks random outputs for each input...
- The PRP (on the right) must ensure outputs are never repeated:
  - $\mathcal{Y}$ tracks all outputs used so far.
  - $\setminus$ means “set difference” - pick from values not in $\mathcal{Y}$
  - $\cup$ means “set union” - add the new value to $\mathcal{Y}$
- This ensures each output appears exactly once - the definition of a permutation.

$\mathcal{L}^{F}_{\text{prp-rand}}$

$L := [\,]; \mathcal{Y} := \varnothing$

$\underline{\text{PRP.QUERY}(X)}$:
if $L[X]$ undefined:
  $Y \twoheadleftarrow \{0,1\}^n \setminus \mathcal{Y}$
  $\mathcal{Y} := \mathcal{Y} \cup \{Y\}$
  $L[X] := Y$
return $L[X]$

PRP versus PRF: the collision distinguisher

Query q distinct inputs and check whether any two outputs coincide.

Random permutation

A collision is impossible.

Random function

A collision can occur among the n-bit outputs.

$$\Pr[\text{collision}] \leq \binom{q}{2} 2^{-n} = \frac{q(q-1)}{2^{n+1}}.$$

- Sampling with and without replacement agrees until a repeated output is drawn.
- Around $q \approx 2^{n/2}$, collisions give a substantial distinguishing signal.

The PRP/PRF switching lemma

For an adversary making at most q forward queries to an n-bit permutation:

$$\mathrm{Adv}_P^{\mathrm{prf}}(A) \leq \mathrm{Adv}_P^{\mathrm{prp}}(A) + \frac{q(q-1)}{2^{n+1}}.$$

- First replace the real PRP with a random permutation.
- Then replace the random permutation with a random function.
- The second replacement costs at most the collision bound.

Use condition: choose $q \ll 2^{n/2}$ so the collision term is small.

Source: The Joy of Cryptography, PRP/PRF switching lemma.

Pseudorandom permutation: definition

Pseudorandom Permutation (PRP)

A keyed permutation $F^{\pm}$ is a secure pseudorandom permutation (PRP) if the following two libraries are indistinguishable:

- n is the input length of the PRP. Also the output length!
- λ is the key size and hence the security parameter.

$\mathcal{L}^F_{\text{prp-real}}$

$K \twoheadleftarrow \{0,1\}^\lambda$

$\underline{\text{PRP.QUERY}(X)}$:
return $F(K,X)$

$\approx$

$\mathcal{L}^F_{\text{prp-rand}}$

$L := [\,]$; $y := \varnothing$

$\underline{\text{PRP.QUERY}(X)}$:
if $L[X]$ undefined:
  $Y \twoheadleftarrow \{0,1\}^n \setminus y$
  $y := y \cup \{Y\}$
  $L[X] := Y$
return $L[X]$

Pseudorandom permutation: definition

Pseudorandom Permutation (PRP)

A keyed permutation $F^{\pm}$ is a secure pseudorandom permutation (PRP) if the following two libraries are indistinguishable:

- We obviously can't use $\mathcal{L}^{F}_{\text{prp-rand}}$ in the real world.
- It doesn't scale, so we need practical approximate alternatives.

$\mathcal{L}^{F}_{\text{prp-real}}$

$K \twoheadleftarrow \{0,1\}^{\lambda}$

$\underline{\text{PRP.QUERY}(X)}$:

return $F(K, X)$

$\approx$

$\mathcal{L}^{F}_{\text{prp-rand}}$

$L := [\,];\ y := \varnothing$

$\underline{\text{PRP.QUERY}(X)}$:

if $L[X]$ undefined:

  $Y \twoheadleftarrow \{0,1\}^{n} \setminus y$

  $y := y \cup \{Y\}$

  $L[X] := Y$

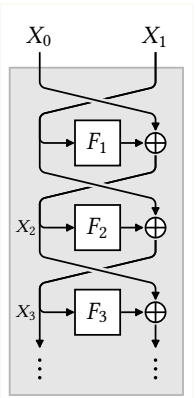
return $L[X]$

Building a permutation through Feistel ciphers

- An r-round **Feistel cipher** with **round functions** $F_1, \ldots, F_r$ is defined as follows:

$\mathbb{F}(X_0 \| X_1)$:
for $i = 1$ to r:
  $X_{i+1} \coloneqq X_{i-1} \oplus F_i(X_i)$
return $X_r \| X_{r+1}$

- A Feistel cipher is always a permutation on $\{0,1\}^{2n}$, regardless of its round functions.



Feistel cipher.
Source: The Joy of Cryptography

Building a permutation through Feistel ciphers

Feistel Ciphers Are Permutations

A Feistel cipher is always a permutation on $\{0,1\}^{2n}$, regardless of its round functions.

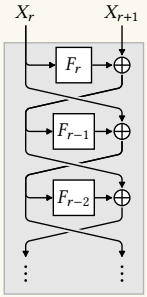
- **Proof:** Each round of a Feistel cipher computes the next block as:
  - $X_{i+1} = X_{i-1} \oplus F_i(X_i)$
- To invert this round, we can rearrange the equation to solve for X_{i-1} in terms of X_i and X_{i+1}:
  - $X_{i-1} = X_{i+1} \oplus F_i(X_i)$
- F_i itself does not need to have an inverse - both the forward and inverse directions of the Feistel cipher evaluate F_i in the forward direction! □

Building a permutation through Feistel ciphers

Inversion

- $\mathbb{F}^{-1}$ is the inverse. This proves invertibility, not PRP security:

$\mathbb{F}^{-1}(X_r \| X_{r+1})$:
for $i = r$ **down to** 1:
  $X_{i-1} \coloneqq X_{i+1} \oplus F_i(X_i)$
return $X_0 \| X_1$



Source: The Joy of Cryptography

Forward and backward on one slide

Just in case it's helpful

$\mathbb{F}(X_0 \| X_1)$:
for $i = 1$ to r:
$X_{i+1} := X_{i-1} \oplus F_i(X_i)$
return $X_r \| X_{r+1}$

$\mathbb{F}^{-1}(X_r \| X_{r+1})$:
for $i = r$ down to 1:
$X_{i-1} := X_{i+1} \oplus F_i(X_i)$
return $X_0 \| X_1$

Keyed Feistel ciphers

$$X_{i+1} = X_{i-1} \oplus F_{K_i}(X_i).$$

- In the standard theorem, round keys are sampled uniformly and independently.
- Merely choosing different key values does not establish independence.
- A key schedule is an algorithm that derives round keys from a master key.
- Practical key schedules need their own analysis; the independent-key theorem does not automatically apply.

Two requirements

We need suitable round functions and enough rounds, in addition to invertibility.

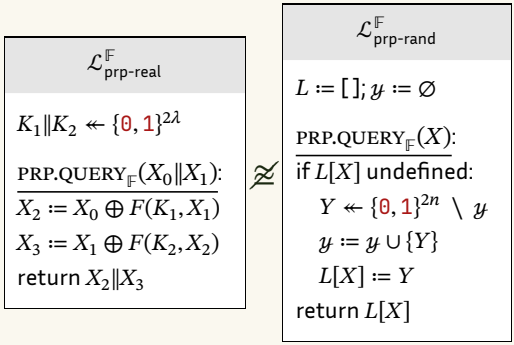
Breaking a 2-round Feistel cipher

- **A 2-round Feistel cipher cannot be a secure PRP.**

$\underline{\mathbb{F}(K_1 \| K_2,\ X_0 \| X_1):}$
$X_2 := X_0 \oplus F(K_1, X_1)$
$X_3 := X_1 \oplus F(K_2, X_2)$
return $X_2 \| X_3$

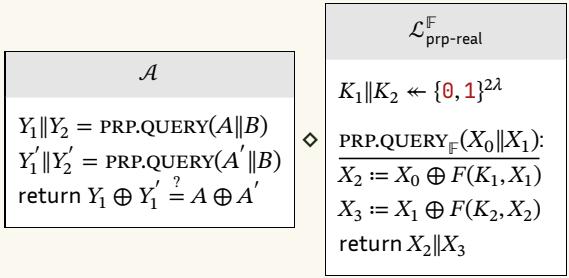
Breaking a 2-round Feistel cipher

- Once again, real or random?
- We can indeed produce an adversary $\mathcal{A}$ that can distinguish between these two libraries.



Breaking a 2-round Feistel cipher

- Let's query $(A\|B)$ and $(A'\|B)$ where $A \neq A'$:
  - $Y_1\|Y_2 = \text{PRP.QUERY}(A\|B)$
  - $Y_1'\|Y_2' = \text{PRP.QUERY}(A'\|B)$
- In a 2-round Feistel cipher:
  - $Y_1 = A \oplus F(K_1, B)$
  - $Y_1' = A' \oplus F(K_1, B)$
  - $Y_1 \oplus Y_1' = A \oplus A'$
- In a uniformly random permutation, $\Pr\left[Y_1 \oplus Y_1' = A \oplus A'\right]$ is negligible.



Three-round Feistel: the security theorem

Luby–Rackoff theorem

If $F : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ is a secure PRF, three Feistel rounds using independent uniform keys give a secure PRP on $\{0,1\}^{2\lambda}$.

- One or two rounds are insufficient for PRP security.
- Defeating the previous distinguisher is not a proof against all distinguishers.
- The theorem supplies that guarantee under its stated assumptions.
- It does not say that any three-round practical cipher is secure.

Source: The Joy of Cryptography, Feistel construction / Luby–Rackoff theorem.

PRP versus strong PRP

PRP

The adversary can query $P_k(x)$. The ideal oracle is a random permutation.

Strong PRP

The adversary can query both $P_k(x)$ and $P_k^{-1}(y)$. The ideal oracles share one random permutation.

- Both definitions require an efficient inverse given the key.
- The difference is whether the attacker can query that inverse.
- Three independent PRF rounds give a PRP, but not a strong PRP.
- Four independent PRF rounds give a strong PRP.

Source: The Joy of Cryptography, strong PRPs / Luby–Rackoff theorem.

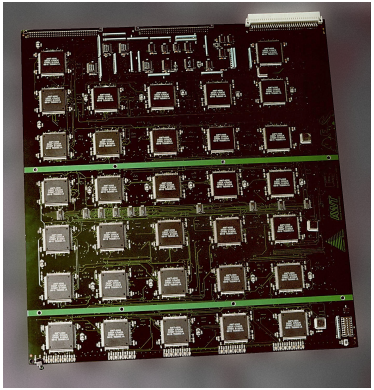
DES: Feistel in practice

- **Data Encryption Standard (DES)** is a classic real-world implementation of the Feistel structure.
- Properties:
  - 16 Feistel rounds.
  - 56-bit key (64 bits with parity).
  - 64-bit block size.
  - Standardized in 1977.
- Problem: By the 1990s, 56-bit keys became too small for security.

- **Triple DES (3DES)** replaced it:
  - Uses three DES operations in sequence.
  - $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$
  - Key length and effective security are different quantities.
  - Compatible with legacy DES when $K_1 = K_2 = K_3$
- NIST disallows TDEA for applying new protection after 2023; legacy decryption is a separate use.
- AES is the modern block-cipher example used here.

EFF's "Deep Crack"

- In 1998, the Electronic Frontier Foundation built a special-purpose machine called "Deep Crack".
- Cost: Only $250,000 (far less than the NSA budget!)
- Purpose: Prove that 56-bit DES keys were insufficient.



EFF's "Deep Crack"

EFF's "Deep Crack"

- July 1998: Deep Crack broke a DES challenge in just 56 hours.
- Message revealed: "It's time for those 128-, 192-, and 256-bit keys."
- Impact:
  - Publicly demonstrated DES was obsolete.
  - Accelerated adoption of AES.
  - Made "it's theoretically breakable" a practical reality.
  - Priceless reaction from government officials!



Paul Kocher. He's still active in cryptography today, with an insanely productive research career and many crazy attacks.

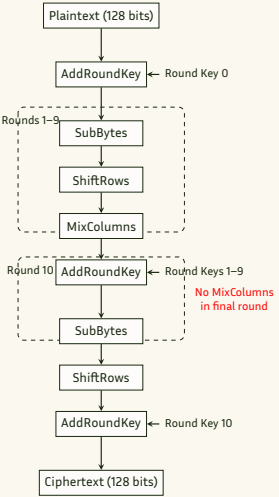
AES: a good example of a PRP

- AES is the most widely used PRP in the world.
- It works on fixed-size blocks: 128 bits.
- Key sizes: 128, 192, or 256 bits.
- Each AES key defines a specific permutation over the space of all 128-bit values.
- For each key, AES maps each possible 128-bit input to exactly one 128-bit output.

- Each key selects a permutation from the AES family.
- AES is efficiently invertible:
  - $\mathsf{Dec}(K, \mathsf{Enc}(K, M)) = M$
- AES is believed to be computationally indistinguishable from a random permutation.
- Has withstood extensive cryptanalysis for over 20 years.

AES-128 structure

- Internal structure: substitution-permutation network with multiple rounds.[a]
  - SubBytes: non-linear substitution.
  - ShiftRows: transposition.
  - MixColumns: mixing operation.
  - AddRoundKey: XOR with round key.



[a] Check out this amazing interactive animation of AES's internal structure: https://formaestudio.com/rijndaelinspector/archivos/Rijndael_Animation_v4_eng-html5.html

AES security: distinguish the attack models

- AES is a practical PRP candidate; its security is supported by cryptanalysis, not an unconditional proof.
- Side-channel attacks target implementations and leakage, rather than the ideal black-box PRP interface.
- Related-key attacks give the attacker capabilities beyond the single-secret-key game.
- Grover search gives a quadratic query speedup: about 2^{64} quantum oracle calls for a 128-bit key.
- Quantum gates, circuit depth, and error correction add substantial cost; oracle calls are not ordinary machine operations.

Source: NIST PQC workshop: practical cost of Grover for AES key recovery.

現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.4: Pseudorandomness

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.